BTS – FILIÈRE DAI

# Gestion de la mémoire et stockage temporaire

Architecture des applications mobiles et gestion de la performance

## Introduction

Toute application moderne manipule des données éphémères. Contrairement au stockage persistant (SQLite), le stockage temporaire vise la rapidité et la réduction de l'empreinte réseau. Ce cours détaille les trois piliers du stockage non-permanent.

# Les 3 Piliers de l'Éphémère

**Cache & RAM**

Le **Cache** accélère les accès futurs aux données lourdes. La **RAM** gère l'exécution immédiate du code.

**Sessions Utilisateur**

Maintient l'état contextuel (connexion, panier) pendant que l'utilisateur navigue dans l'application.

# Le concept de Stockage Temporaire

**0%**

PERSISTANCE

**Données non conservées définitivement**

Le stockage temporaire sert à conserver des informations pendant une durée limitée. Contrairement aux bases de données, ces données disparaissent après usage ou fermeture.

**Avantages :** Vitesse d'accès ultra-rapide, économie de stockage disque, et réduction des appels réseau.

# Pourquoi utiliser un Cache ?

**Éviter le calcul**

Ne pas recalculer des algorithmes lourds si le résultat est déjà connu.

**Économie Réseau**

Ne pas retélécharger les mêmes images ou fichiers JSON inutilement.

**Expérience Fluide**

Affichage instantané des éléments déjà consultés par l'utilisateur.

# Cache : Détails techniques

- **LruCache :** Algorithme "Least Recently Used" supprimant les éléments les moins consultés.
- **HTTP Cache :** En-têtes standards permettant au navigateur de stocker les ressources.
- **Images :** Mise en cache des Bitmaps pour une fluidité lors du défilement des listes.
- **Nettoyage :** Le système peut purger le cache si l'espace disque devient insuffisant.

Cache Memory

# Implémentation du LruCache (Java)

```java
// Définition d'un cache de 4Mo pour des chaînes de caractères;
int cacheSize = 4 * 1024 * 1024;
LruCache myCache = new LruCache(cacheSize);
// Ajouter une donnée;
myCache.put("produit_102", "Détails du produit JSON");
// Récupérer une donnée;
String data = myCache.get("produit_102");
```

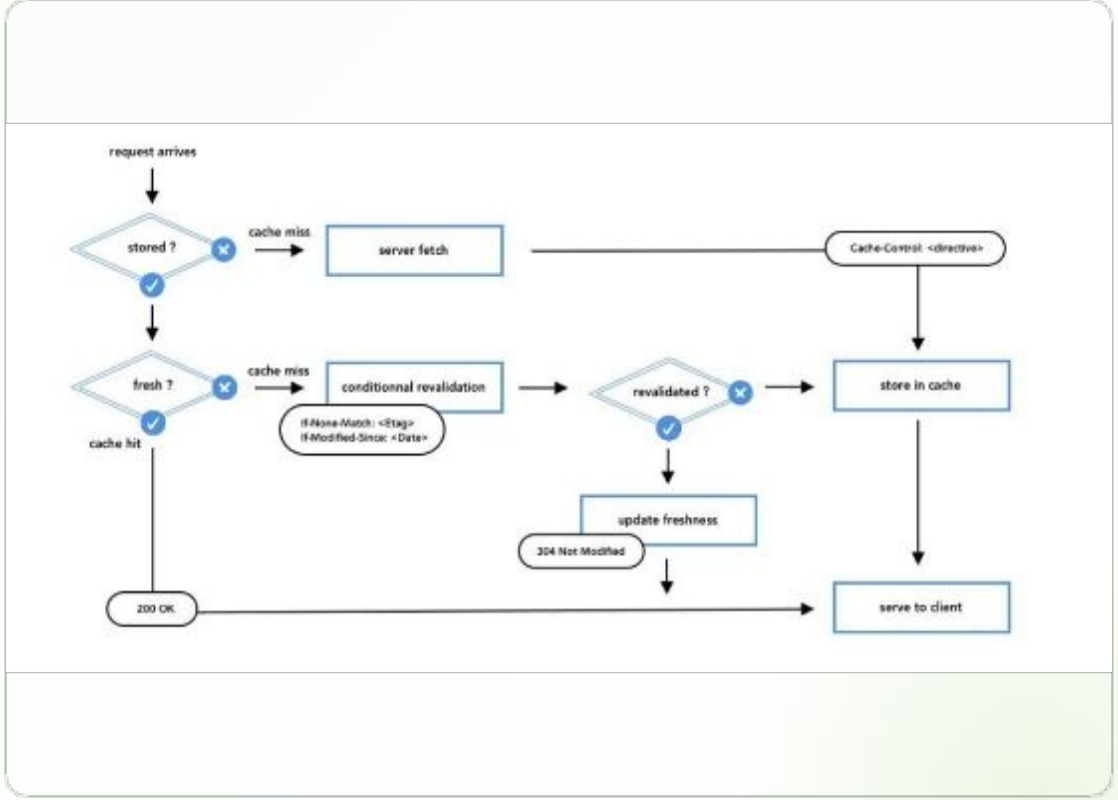
# Fonctionnement du Cache (Flux)

Lorsqu'une donnée est demandée :

1. L'application vérifie si la donnée existe dans le **Cache**.

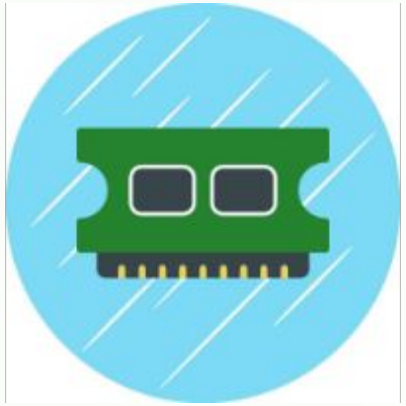
2. Si oui (*Cache Hit*), elle est renvoyée instantanément.

3. Si non (*Cache Miss*), elle est téléchargée depuis le serveur, affichée, puis stockée dans le cache pour la prochaine fois.



## 2. La Mémoire Volatile (RAM)

La RAM est le cerveau immédiat de votre application. Elle stocke les variables actives pendant l'exécution.



# Qu'est-ce que la RAM ?

**100%**

VOLATILE

**Stockage pendant l'exécution**

La **Random Access Memory** stocke les variables, les objets et les instructions machine pendant que l'application est active.

⚠️ **Attention :** Tout disparaît si l'application est tuée par le système ou si le téléphone redémarre.

# Durée de vie en RAM

**0**

PERSISTANCE

## Cycle de vie critique

Toute donnée stockée uniquement dans une variable Java est perdue dès que le processus système est tué ou que l'application est fermée par l'utilisateur.

**Impact :** Nécessité de sauvegarder l'état (Bundle ou SQLite) avant l'arrêt.

# Variables et Volatilité

Exemple de données volatiles :

```
public class UserActivity extends Activity {

    // Variable stockée en RAM
    private String username = "Ahmed";

    @Override
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);

        // Utilisation de la variable
        showWelcome(username);
    }
}
```

## 3. Sessions Utilisateur

Garder le contexte de navigation de l'utilisateur.

# Gestion des Sessions

**Authentification**

Conserver le Token JWT ou l'ID de l'utilisateur pour rester connecté.

**Panier Temporaire**

Garder les articles sélectionnés avant la validation finale.

**Préférences**

Conserver des filtres ou des réglages d'affichage temporaires.

# Gestion des Sessions

**Authentification**

Conserver le Token JWT ou l'ID de l'utilisateur pour rester connecté.

**Panier Temporaire**

Garder les articles sélectionnés avant la validation finale.

**Préférences**

Conserver des filtres ou des réglages d'affichage temporaires.

# Le Pattern Singleton

**Objectif :** S'assurer qu'une classe n'a qu'une seule et unique instance dans toute l'application et fournir un point d'accès global.

**Cas d'usage :**

- Gestionnaire de base de données.
- Stockage de session utilisateur.
- Journalisation (Logger).

**Structure :**

1. Constructeur private.
2. Instance statique de la classe.
3. Méthode statique getInstance().

# Le Pattern Singleton

On ne veux pas avoir :

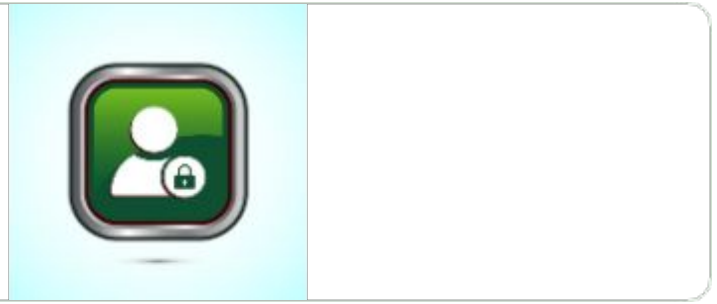
user1
user2
user3
Interdiction de  plusieurs sessions

Singleton garantit :

UNE seule session

# Code : Session Manager

Pour gérer une session globalement, on utilise souvent le pattern **Singleton** en Java Android.



```java
public class SessionManager {

    private static SessionManager instance;
    private String utilisateur;
    private SessionManager() {} // empêche new

    public static SessionManager getInstance() {
        if (instance == null)
            instance = new SessionManager();
        return instance;
    }
    public void setUtilisateur(String u) {
        utilisateur = u;
    }
    public String getUtilisateur() {
        return utilisateur;
    }
    public void logout() {
        utilisateur = null;
    }
}
```

# Gestion des Sessions (Java)

```
public class SessionManager {
    // Singleton pour accès global;
    private static SessionManager instance;
    private String userToken;
    private boolean isLogged = false;
    // Stocker la session;
    public void startSession(String token) {
        this.userToken = token;
        this.isLogged = true;
    }
}
```

*Note : Les variables de session vivent tant que l'instance de l'application est en mémoire.*

# Résumé du Stockage Temporaire

| Type | Durée de vie | Emplacement | Exemple |
|---|---|---|---|
| **Cache** | Temporaire (Purgeable) | Disque / RAM | Images téléchargées |
| **RAM (Volatile)** | Tant que l'app tourne | Mémoire vive | Variables Java |
| **Session** | Durée de connexion | Singleton / RAM | Utilisateur connecté |

# Temporaire vs Local vs Cloud

| Critère | Temporaire | Local (SQLite) | Cloud (Firebase) |
|---|---|---|---|
| **Persistance** | Nulle | Élevée (Définitive) | Maximale |
| **Accès** | Instantané | Rapide | Lent (Réseau) |
| **Coût** | Faible | Moyen (Disque) | Élevé (Data) |

# MERCI !

Des questions sur le stockage temporaire ?